TESTABLE WHITEBOX . TOOL VALIDATION AUTOMATION

Workflow Guide

End-to-end flow of the dev branch scripts, file by file, and how anyone on the team runs them step by step

BRANCH	VERSION	WRITTEN ON	STACK
dev	**a78fbb1 + working copy**	**29 Oct 2026**	**Playwright, TypeScript, Node 22**

What is inside

1 The big picture
what the scripts do in one minute

2 End-to-end flow
one picture from setup to Teams, and what is where

3 Pipeline and timing
one account and two accounts

4 Main flow
the 15 browser steps

5 Background worker
hand-off, S3 check, SharePoint, Teams

6 Two-account mode
npm run parallel, lanes and rules

7 Step by step for a new teammate
access, install, .env, first run, every run

8 File map
which file does which job

9 .env reference
every setting, what to put, defaults

10 Result folders
what each output file is

11 Troubleshooting
symptom, cause, fix

12 QA EKS Mentor (AI log analysis)
the local model, how it plugs in, how to set it up

13 Cheat sheet
commands and words in one page

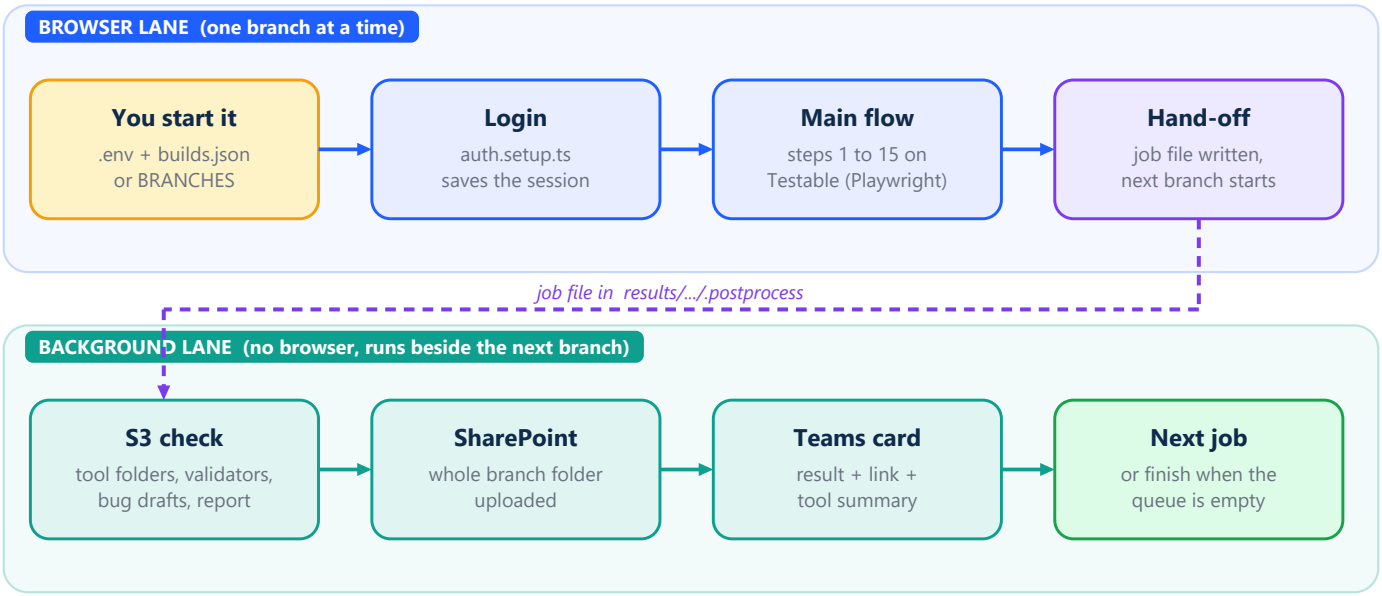
Who this is for
Anyone who runs the scripts, reads their results, or needs to change them. Sections 1 to 3, 6, 7, 9, 10, 11 and 13 need no code knowledge. **Section 7 is the checklist for a new teammate.** Section 8 is for people who open the files.

1 The big picture

what the scripts do in one minute

The scripts test a branch on the **Testable** platform exactly like a person would, then check what the platform wrote to **AWS S3**, and send the result to the team. You give them one or more **GitHub branch links**. For every branch they do this:

- **Run the branch on Testable** with a real browser (Playwright): connect the repository, pick the branch, press Execute, wait for the analysis to finish, and take screenshots and the taxonomy report.
- **Check the evidence in S3**: for every technique open the tool's folder, read the files, and say **PASS**, **EMPTY** or **FAIL**. Language validators judge whether each tool behaved correctly for that language and version.
- **Write bug drafts** (PDF, screenshot, Jira text) for the problems that belong to the platform. Nothing is sent to Jira automatically.
- **Upload the whole branch folder to SharePoint** and **post a card in Teams** with the result and a link.

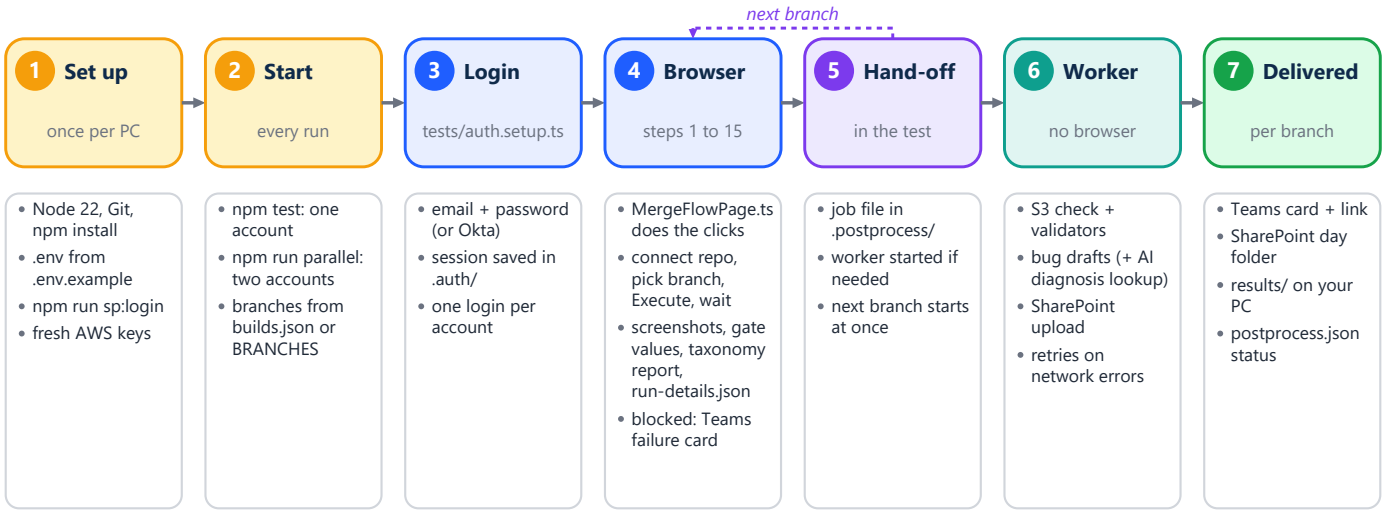


The key idea of this branch: after the browser part of a branch is finished, everything that does not need the browser (S3 check, SharePoint, Teams) moves to a **background worker**. The next branch starts on the platform right away, so the platform is never waiting for your reports to be written. With **two Testable accounts** two branches can run on the platform at the same time (section 6).

Three things to remember
1. A browser lane runs **one branch at a time**. 2. The background lane works on branch N **while** branch N+1 is executing. 3. Every setting lives in **.env** (section 9). A value in .env always wins over a value typed in the terminal.

2 End-to-end flow

from the first setup to the Teams card, in one picture



Reading the picture. Stage 1 is done once per PC. From stage 2 on everything repeats for each branch. After stage 4 the branch is "passed" as far as the browser is concerned; stages 6 and 7 finish while the next branch is already in stage 4.

What is where right now

Part	Status	Notes
Browser flow, background worker, SharePoint upload, retries, live log, coverage-preview early stop	**pushed to origin/dev**	The reliable core. Everything in sections 3 to 5.
Two-account mode (npm run parallel, ACCOUNT switch), small guarded switches, README and .env.example notes	**working copy, not pushed**	Tested with a stand-in script, not yet with a real second account. Check that **npm run** lists "parallel" before you use section 6.
QA EKS Mentor: the local AI model, qa-eks-mentor/, utils/eksMentor.ts, a call inside bugReport.ts	**model running here; hook in working copy**	The model qa-eks-mentor-trained is installed in Ollama on this PC. Bug drafts now print its diagnosis **when a ready analyze.py report exists** for that tool and run (section 12).

The order of every run

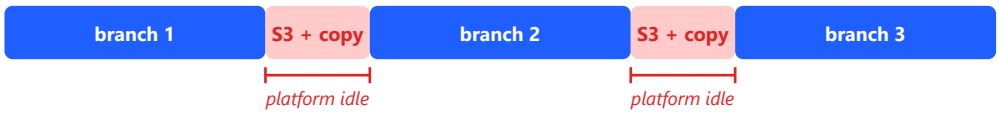
#	Moment	What happens	Where you see it
1	Start	Branches are read from **BRANCHES** or **builds.json**. The run folder **results/merge-flow_<date>_<time>** is created (two accounts: one folder per lane, ending _acct1 and _acct2).	terminal
2	Login	**auth.setup.ts** logs in with the Testable account from .env and saves the session.	test "auth.setup.ts"
3	Branch N, browser	Main flow steps 1 to 15 (section 4).	test "Whitebox run - <branch>"
4	Hand-off	When branch N passes, a job file is written and the worker is started (if not already running). Branch N+1 starts.	"handed to the background worker"
5	Branch N, worker	S3 check, bug drafts, SharePoint upload, Teams card (section 5). Runs beside branch N+1.	lines **[post] [branch]**
6	Logout	After the last branch the browser logs out of Testable.	test "Logout"
7	Wait for the worker	The last branch's post-processing finishes; the worker ends.	"Background post-processing finishes"

3 Pipeline and timing

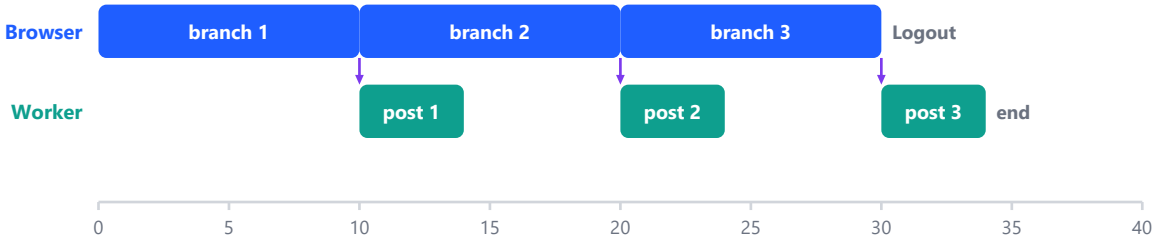
why the platform no longer sits idle

One account. Before, the script finished all the reports of a branch before it started the next one, so the platform waited. Now the reports are made by the worker beside the browser.

BEFORE
inline, one after the other

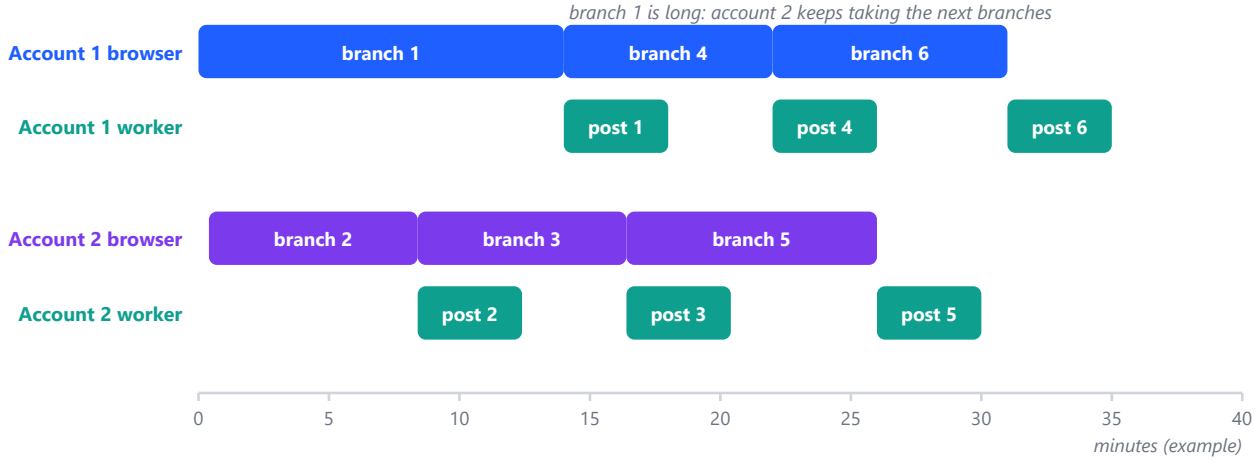


NOW
background worker



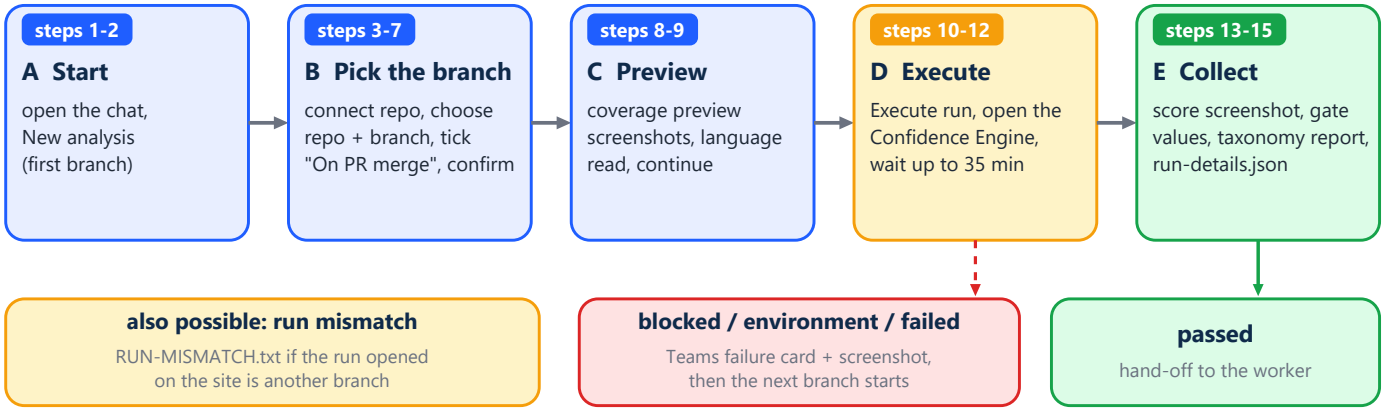
The red gaps in the top lane are the time the platform sat idle. In the bottom lane the browser starts branch 2 straight after branch 1, and the worker handles branch 1 in the background. Only the **last** branch's post-processing happens after the last execution ends. The minutes are examples; real runs take about 10 minutes per branch and about 3 to 5 minutes of post-processing.

Two accounts. With a second Testable account a second lane runs at the same time. A lane that finishes takes the next waiting branch, so a long branch on one account does not hold the others back.



4 Main flow

the 15 steps the browser does for each branch



Which steps stop a branch
Steps **1, 3, 4, 5, 6, 7, 9, 10, 11 and 12** are *critical*: if one fails, this branch stops and the next branch starts. Steps 2, 8, 13, 14 and 15 are recorded but do not stop the branch on their own; a failure there still marks the branch failed at the end.

Step	What the script does	Function in pages/MergeFlowPage.ts	Saved as
0	Optional: with **DUMMY_PR=yes** it pushes a comment line to the branch's README so the platform sees a new commit.	utils/dummyCommit.ts	-
1	Opens the Chat Execution Assistant (later branches use the sidebar chat icon).	openChat	evidence/<branch>_1_PASS.png
2	First branch only: "Can I safely deploy this commit?" then New analysis / Refresh repos.	startNewAnalysis	-
3	Change repository / branch, then Connect or Import repository. Waits for the repo sync (cap 14 min).	connectRepo	-
4	Clicks the repository, then Configure repository.	chooseRepo	-
5	Selects the branch: "+N branches", search, tick, Done.	chooseBranch	-
6	Makes sure the "On PR merge" trigger is ticked.	ensurePrMergeTicked	evidence screenshot
7	Confirms the repository. The site may answer "Branch unavailable": the branch is then **blocked**.	confirmRepo	errors/site-error.txt (if blocked)
8	Coverage preview: repo structure, what will be analysed, uncovered areas. Reads Build type, Language and Language version. **Stops within about 10 seconds** when the site says "Analysis is blocked on ...", and waits at most 1 minute per section.	coveragePreview, waitSectionOrBlocked, saveRepoDetails	01_ 02_ 03_ screenshots, repo-details.json
9	Continues from the preview / Review branches. Stops if the site blocks the analysis.	continueFromReview	-
10	Run plan, then Execute run. Retries if the run does not start.	executeRun	evidence screenshot
11	Execution status, then Open Confidence Engine.	openConfidenceEngine	-
12	Waits for the analysis to finish (**RUN_TIMEOUT_MIN**, default 35 minutes).	waitForRunToFinish	05_execution_status.png
13	Screenshot of the PR Confidence Analysis score.	screenshotPrScore	04_pr_confidence_score.png
14	Reads every technique x classification gate value from the page. With **GATE_SHOTS=on** it also takes a screenshot of each gate.	captureUiScore	ui-values.json, gates/
15	Downloads the taxonomy report, builds the report with the UI value column, saves the run record. Checks that the run on the site is the branch you asked for.	downloadTaxonomyReport, saveRunRecord, mergeTaxonomy()	taxonomy-gate-<run>.html, ...-with-ui-values.html, run-details.json

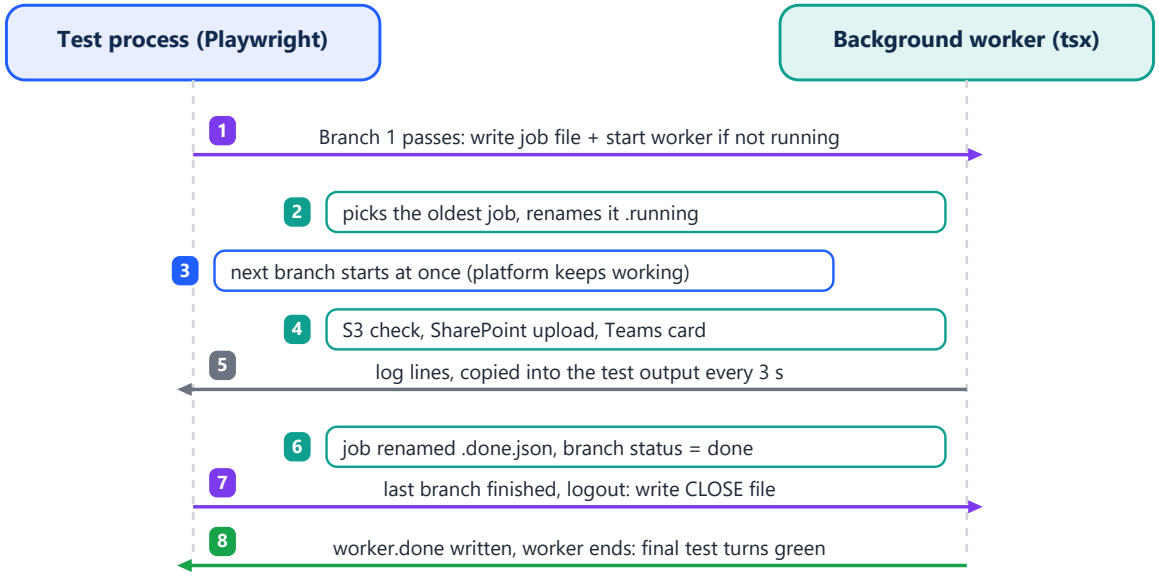
When a branch does not pass
A failed, blocked (site message) or environment problem (site down, repo sync stuck) ends that branch only. A screenshot and the network log are saved in **errors/**, a failure card goes to Teams straight away, and the next branch starts. A branch that failed has no background post-processing, because there is no run to check.

5 Background worker

what happens to a passed branch after the browser is done

5.1 The hand-off

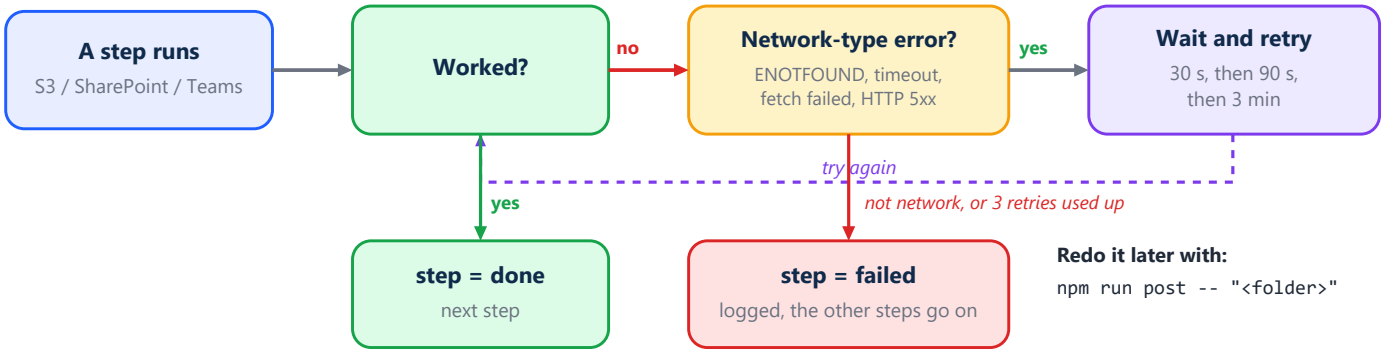
The browser test and the worker talk through plain files in **results/<run>/.postprocess/**, so nothing is lost if either side stops. This is the conversation for a run with several branches:



File in .postprocess/	Meaning
<time>-<branch>.job.json	a branch waiting to be post-processed
...job.json.running	the worker is working on it now (renamed back to .job.json if the worker dies, so the next worker redoes it)
...done.json / ...failed.json	finished jobs
worker.pid, CLOSE, worker.done	the worker's registration, "no more jobs are coming", and "I have finished"

5.2 What the worker does for each branch

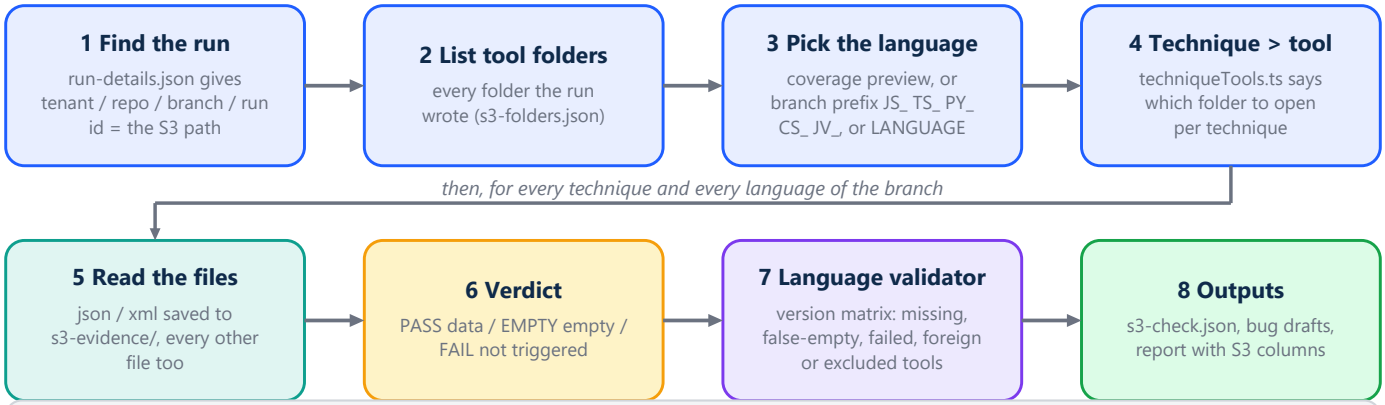
Order	Step	Details	If it fails
1	**S3 check**	Needs AWS keys in .env. Reads the run's tool folders, judges every tool, writes s3-check.json, bug drafts and the report with S3 columns (section 5.3).	logged; SharePoint and Teams still run
2	**SharePoint**	Uploads the whole branch folder to Automation Evidences Dump / date / branch (section 5.4).	logged; Teams still runs
3	**Teams**	Posts the success card with the evidence link and the tool summary (section 5.5).	marked failed, redo with npm run post



Network problems (DNS, timeouts, HTTP 5xx, "fetch failed") are retried three times after 30 seconds, 90 seconds and 3 minutes. Every branch has a status file **postprocess.json** with *queued, running, done* or *failed* for S3, SharePoint and Teams.

5.3 The S3 check, explained

Every tool the platform runs writes its files under one S3 path built from the run: **development/cell-001/<tenant>/<project>/<repository>/<branch id>/<branch>/<commit>/<run id>/<tool>/...**. The check follows eight steps:



The secondary tool is used only when the primary tool's folder is missing. A "primary N/A" row checks only its secondary tool.
When two languages share a folder (for example lizard), the file names inside (.js .ts .py .cs .java) say which language a file is for.

Verdict	Meaning	Typical reason
PASS	The tool was triggered and its file holds data.	e.g. lizard.xml with 123 items
EMPTY	Triggered, but the file is empty or the tool said "not applicable / skipped".	status=not_applicable, baseline_missing, empty list
FAIL	Not triggered: no folder, no json/xml file, or only a failure.json.	no folder jacoco in this run
N/A	The tool does not apply to this row.	primary tool not applicable for this classification

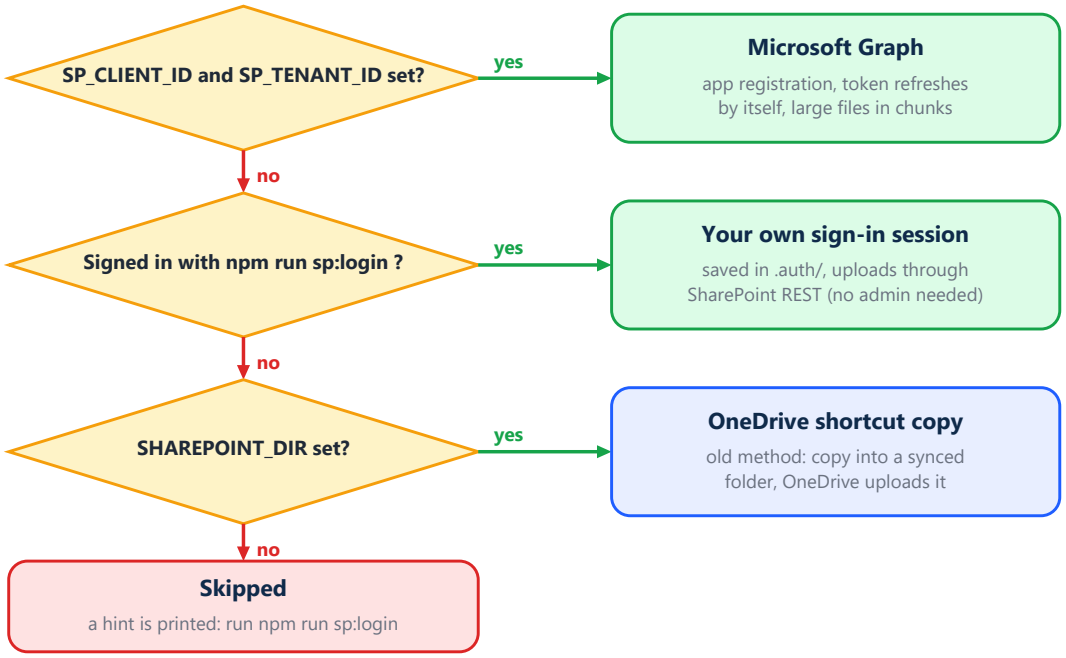
Language	Validator file	What it judges
Python	utils/pyValidate.ts	tool list per Python version, what each tool's file must hold
JavaScript	utils/jsValidate.ts	tool list per Node version, package manager and layout
TypeScript	utils/tsValidate.ts	same as JavaScript, with the TypeScript tool names
C#	utils/csValidate.ts	tool list per .NET version and target
Java	utils/javaValidate.ts	tool list per JDK version and build tool

The validators produce **defect tags** for platform problems: *[Planning Defect]* (a tool that should run is missing, or one that must be excluded ran), *[False Empty Defect]* (empty report after scanning 0 files), *[Runner Execution Defect]* (the tool crashed) and *[Language Contamination]* (a tool of another language was planned). Failures caused by the customer's own repository are listed as repo-side and are **not** filed as platform bugs. For each platform defect **bugReport.ts** writes a 2-page PDF, a screenshot of the report row and a Jira text file into **bugs/**. A local AI model can

add a root-cause diagnosis to each bug draft (section 12).

5.4 SharePoint

The upload target is the team folder **Automation Evidences Dump** on the Testable-ExecutionTeam site. The script creates a folder for the day (**DD-MM-YYYY**) and one folder per branch inside it. A second run of the same branch name on the same day goes to **<branch>_RUN2**, then **_RUN3**. Three upload methods exist; the script picks the first one that is set up:



Where it lands

Automation Evidences Dump / DD-MM-YYYY / <branch>

Same name again the same day: <branch>_RUN2, _RUN3 ...

Sign in once
Each developer runs **npm run sp:login** one time. A browser opens; sign in with your Microsoft work account and approve the prompt. The session is saved in .auth/ and reused. If your company's policy ends the session, the upload says "sign-in expired" and you run the command again. Files larger than 240 MB are not uploaded and are listed in sharepoint-copy-errors.txt.

5.5 Teams

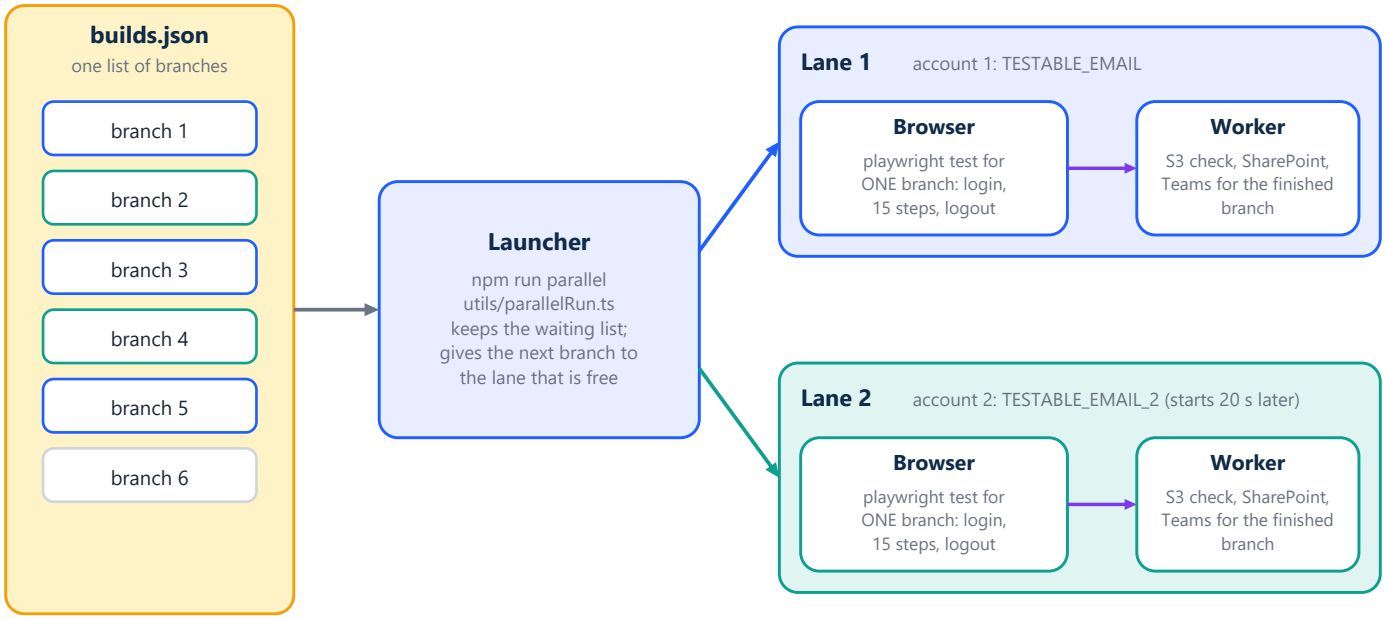
Card	When	Contents
Success	sent by the worker, after S3 and SharePoint	Branch name, repo, environment (QA/DEV), minutes; buttons "Open the run" and "Open evidence folder"; per language: tools expected and OK, tools with a bug (draft reports saved) or "No tool bugs found".
Failure	sent by the browser test at once	Branch name, status (failed / blocked by the site / environment problem), the reason, and a small screenshot.
Did not start	sent by the queue server	A round stopped before a branch could begin (for example the site is down at login).

6 Two-account mode

npm run parallel: the next branch goes to whichever account is free

Status
This mode is in the **working copy and not pushed to origin/dev yet**. It was tested with a stand-in script, not yet with a real second account on Testable. Check that **npm run** lists **parallel** before you use it. Everything else in this guide works without it.

With two Testable accounts you can run two branches on the platform at the same time. Put all branches in **builds.json**, put both logins in **.env**, and start one command. Each account is a **lane**.



How it works

#	What happens
1	**npm run parallel** reads the branch list (builds.json or BRANCHES) and the accounts in .env: **TESTABLE_EMAIL** is account 1, **TESTABLE_EMAIL_2** is account 2 (and so on, up to 9).
2	Each account becomes a lane with its own results folder (**..._acct1**, **..._acct2**), its own saved session (.auth/user.json, .auth/user-2.json) and its own background worker. Lane 2 starts 20 seconds after lane 1 (**PARALLEL_STAGGER_SEC**).
3	A lane takes the next waiting branch and runs it as one normal **playwright test** process (login, 15 steps, logout) with that account. When the branch passes, the lane's worker starts the S3 check, SharePoint upload and Teams card while the lane already takes its next branch.
4	The lane that finishes first takes the next waiting branch. Each branch is taken by only one lane, so the branches are always different.
5	If a branch process ends without a result (wrong password, site down), that branch is tried on the other account. A lane that cannot start two branches in a row stops; the other lane carries on.
6	At the end the launcher tells each worker "no more jobs", waits until the last Teams card and SharePoint upload are done, and prints a summary (branch, account, status, minutes). It also saves it as **results/parallel_<date>_<time>_summary.json**.

Rules and options

Setting	What it does
TESTABLE_EMAIL_2, TESTABLE_PASSWORD_2	The second account (DEV site: TESTABLE_EMAIL_DEV_2 and TESTABLE_PASSWORD_DEV_2). Type it into .env yourself.
LANES=1 (terminal)	Use only the first account.
PARALLEL_STAGGER_SEC (terminal)	Seconds between lane starts. Default 20.

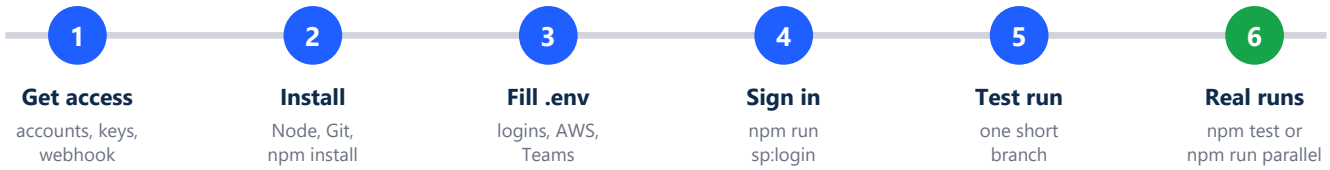
Setting	What it does
ACCOUNT	Do **not** set it for npm run parallel; the launcher sets it per lane. It exists so you can also start a second terminal by hand: **$env:ACCOUNT="2"** then npm test, with different branches. Set it in the terminal, never in .env.
Where to run	In a terminal. Not from the Test Explorer. npm test and the single-account flow are unchanged.

What you see: two Chromium windows (one per lane, the second appears about 20 seconds later; snap them side by side with Win+Left and Win+Right), terminal lines prefixed **[acct1]** and **[acct2]**, and the workers' lines prefixed **[acct1 post]** and **[acct2 post]**. One Teams card per branch; the card does not name the account. SharePoint gets all branches in the same day folder.

Before the first real run
Check by hand that Testable lets both accounts run at the same time. Both accounts need the same GitHub connection so any branch can run on either. Start with two short branches and watch the first two minutes: you should see both browsers working and the line "2 account lane(s): acct1, acct2".

7 Step by step for a new teammate

from nothing to a finished run



7.1 Get access (ask once)

You need	Where to get it	Used for
Node.js 22 and Git on your PC	nodejs.org and git-scm.com	running the scripts
Access to the project's GitHub repository (branch **dev**)	your team lead	getting the code
A Testable account for the QA site (a second one for two-account mode)	your Testable admin or team lead	the browser login (TESTABLE_EMAIL ...)
AWS access: role **QA-S3-Readonly**	AWS access portal (testable.awsapps.com), account foundation-shared-services; ask the AWS admin if the role is missing	the S3 check (three temporary keys)
A Teams channel and its webhook	In the channel: Workflows, "Send webhook alerts to a channel", copy the URL	the Teams cards
Edit rights on the SharePoint folder **Automation Evidences Dump** (Testable-ExecutionTeam site)	the site owner	the evidence upload
The GitHub branch links you want to test	the developers	the work list
Optional: a GitHub token	GitHub, Settings, Developer settings	bug drafts on private repos (GITHUB_TOKEN)

7.2 Install

#	Do this	Command
1	Clone the repository and switch to the dev branch.	`git clone <repo-url>` `git checkout dev`
2	Install the packages (also installs tsx, which runs the background worker).	`npm install`
3	Install the browser Playwright uses.	`npx playwright install chromium`
4	Create your private settings file.	`copy .env.example .env`

7.3 Fill .env (the file in the project folder)

Open **.env** in the editor and fill in these lines. One setting per line, **NAME=value**, no spaces around the equals sign and no quotes. Never share this file and never commit it (git ignores it).

Line	What to put
TEST_ENV	qa (the normal site) or dev
TESTABLE_EMAIL, TESTABLE_PASSWORD	your Testable QA login
TESTABLE_EMAIL_2, TESTABLE_PASSWORD_2	only for two-account mode: the second account
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN	AWS access portal, account foundation-shared-services, role QA-S3-Readonly, "Access keys": copy the three values. **They expire after hours to about a day, so renew them before each run day.**
TEAMS_WEBHOOK_URL	the webhook URL of your Teams channel (empty = no Teams cards)
POSTPROCESS, GATE_SHOTS, DUMMY_PR	leave the defaults: background, off, no

7.4 Sign in to SharePoint once

Run `npm run sp:login`. A browser opens on the SharePoint folder. Sign in with your Microsoft work account and approve the prompt. The window closes by itself and your session is saved in .auth/. Do this once per PC; repeat it only when the upload says "sign-in expired".

7.5 Your first test run (one short branch)

#	Do this	You should see
1	Check the Testable login: `npm run setup`	test "log in" passes; .auth/user.json appears
2	In PowerShell set one short branch: `$env:BRANCHES="https://github.com/OWNER/REPO/tree/BRANCH"`	-
3	Start: `npm test`	a Chromium window opens; the 15 steps run; about 10 minutes
4	When the branch passes, watch the output	"handed to the background worker", then lines starting **[post] [branch]**
5	Wait for the last test	"Background post-processing finishes" turns green
6	Check the three results	a Teams card; a new folder in SharePoint under today's date; the folder results\merge-flow_... on your PC

7.6 Every run after that

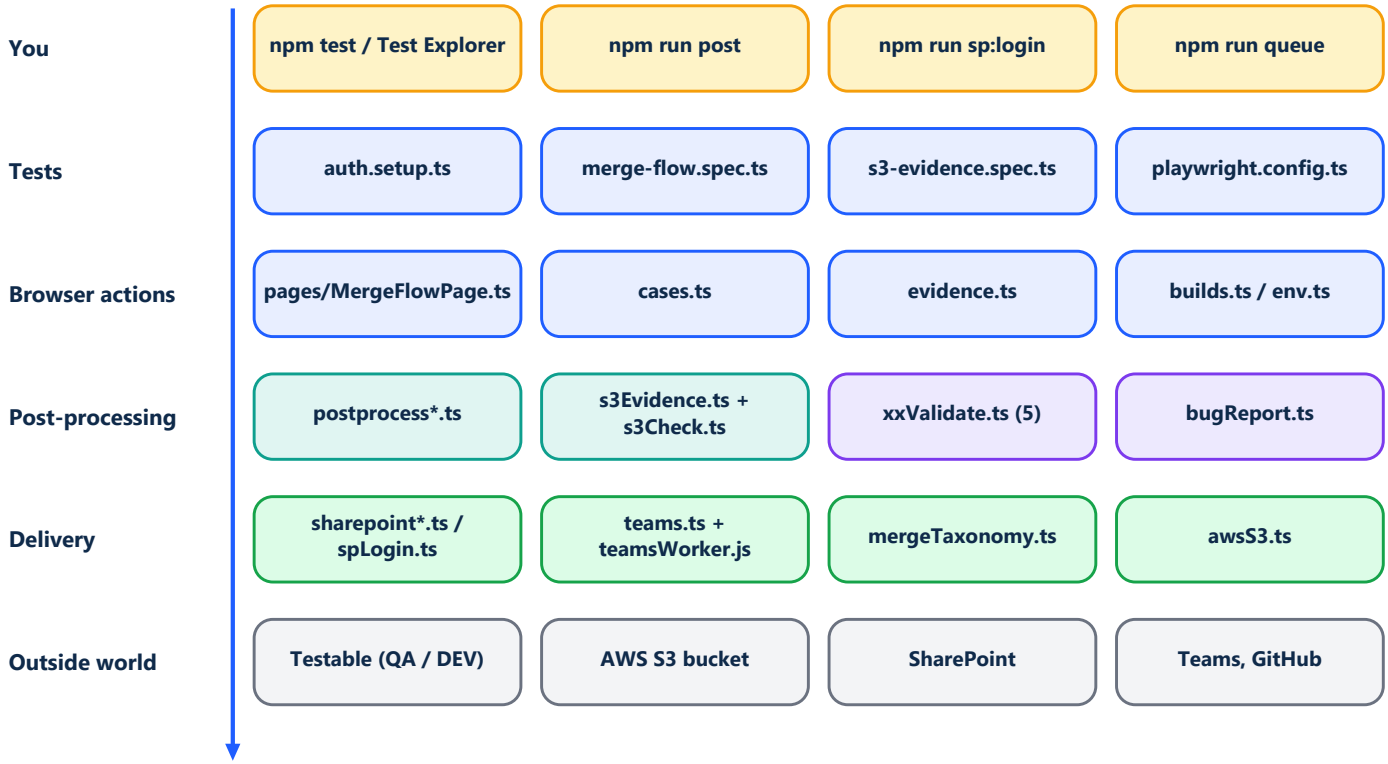
#	Do this	Why
1	Renew the AWS keys in .env if they are older than a day.	otherwise the S3 step fails with ExpiredToken
2	Put your branches in **builds.json** ({ "branches": ["link1", "link2"] }) or set **BRANCHES** in PowerShell (links separated by commas).	the work list
3	Start it: **npm test** (one account) or **npm run parallel** (two accounts, section 6). Or in the Test Explorer: refresh, turn on "Show browser", select the **flow** project, run the whole project.	one command per run
4	Leave the PC alone: do not close the browser windows, do not edit .env or playwright.config.ts, do not lock or shut down the PC.	a running test cannot be paused
5	Watch the **[post] [branch]** lines. When the last test turns green (or the parallel summary prints), everything is done.	live progress
6	Check Teams, SharePoint and the results folder (section 10).	the deliverables
7	Afterwards run **git checkout builds.json**.	so your test list is not committed

7.7 Something went wrong, or you came back later

Situation	Do this
S3, SharePoint or Teams failed for a branch (expired keys, network)	Fix the cause (new keys in .env, or npm run sp:login), then: `npm run post -- "results\...\<branch folder>" --repo owner/name`
A branch was blocked or failed	Read errors\site-error.txt and the Teams card; run that branch again later
You pulled new code	`git pull` then `npm install`
You want to see what a branch's post-processing did	open <branch folder>\postprocess.json and the run folder's postprocess.log

8 File map

which file does which job



Each layer calls the one below it. Read the picture top to bottom to see how a command you type ends up as a Teams card. The table below lists every file in the folder.

File	What it does
Project root	
package.json / package-lock.json	npm scripts (test, setup, s3, queue, sp:login, post, parallel) and packages (Playwright, AWS SDK, exceljs, tsx, dotenv).
playwright.config.ts	Three projects: setup (login), flow (the branch tests, needs setup) and s3 (old re-check). One worker, no retries, a failed branch never stops the run, headed browser, timeouts. With ACCOUNT set: its own report and output folders.
tsconfig.json	TypeScript settings (strict).
builds.json	The branch list. The queue server rewrites it; keep it empty in git.
.env / .env.example	All settings (section 9). .env is private and git-ignored; .env.example is the template.
.gitignore	Keeps .env, .auth/, results/, queue data, logs and the per-account output folders out of git.
README.md, Workflow_Guide_dev_branch.pdf	This documentation. The README is the short version.
testable-merge-flow-HANDOFF.md	Older hand-over notes. Parts are out of date; this guide replaces it.
JIRA_Description_dev_branch_changes.md	A ready-to-paste Jira description of the changes on dev (local file).
Tool_Execution_Results.xlsx	The reference workbook the S3 rules were built from (JS, TS, Python, C# tool lists).
Executive_Architecture_Review.pdf, Testable_Whitebox_Automation_E2E_Flow_and_Operator_Guide.pdf, QA_EKS_Mentor_Architecture_and_Methodology.pdf, generate_e2e_manual_pdf.py, temp_workflow_dump.txt	Other documents and a helper that generated one of them. They are not used by the scripts.
tests/	

File	What it does
auth.setup.ts	Logs in to Testable (normal email + password, or Okta SSO with LOGIN_MODE=okta) and saves the session in .auth/.
merge-flow.spec.ts	The main flow: one test per branch (steps 1 to 15), then Logout, then the test that waits for the worker.
s3-evidence.spec.ts	Stand-alone S3 re-check of an old results folder (npm run s3).
pages/	
MergeFlowPage.ts	Every action on the Testable site: opening the chat, connecting the repo, choosing the branch, Execute, waiting, reading the gates, downloading the report, logging out. Detects "blocked" and "environment" problems and stops early when the site blocks the analysis.
utils/ - running the browser flow	
env.ts	Loads .env, picks QA or DEV and the account (ACCOUNT), builds the site URL, S3 bucket name and the saved-session path.
builds.ts	Reads the branches from BRANCHES, REPO or builds.json (accepts full GitHub /tree/ links).
cases.ts	Runs each step as a named test step, records pass/fail, and stops the branch only on critical steps.
evidence.ts	The run folder, step screenshots, summary.json, and the network log of failed requests.
dummyCommit.ts	Optional README comment commit so the platform analyses a fresh commit (DUMMY_PR).
parallelRun.ts	The two-account launcher (npm run parallel): lanes, the shared waiting list, summary. Working copy, not pushed yet.
utils/ - S3 check and validators	
awsS3.ts	Reads the S3 bucket with the AWS SDK (no browser).
s3Evidence.ts	The S3 check for one branch (the eight steps in section 5.3).
s3Check.ts	Turns a tool's json/xml into PASS / EMPTY / FAIL with a plain-English note.
techniqueTools.ts	The map: technique > primary tool > secondary tool, per language (JS, TS, C#, Python, Java).
pyValidate.ts, jsValidate.ts, tsValidate.ts, csValidate.ts, javaValidate.ts	The per-language validators and their version matrices; they produce the defect tags.
mergeTaxonomy.ts	Builds taxonomy-...-with-ui-values.html: adds the UI value column and the S3 Tool / JSON and S3 comment columns, and a header box with the S3 path and the validation summaries.
bugReport.ts	Bug drafts per platform defect: PDF, report-row screenshot, Jira text, bugs.json. In the working copy it asks eksMentor.ts for an AI diagnosis and prints it in the PDF and Jira text when a ready report exists (section 12).
eksMentor.ts	The bridge to the QA EKS Mentor model: reads a ready report file (folder EKS_MENTOR_DIR) or asks the local Ollama model, and marks which one it was. Never stops the run; returns nothing when Ollama is offline.
utils/ - background worker	
postprocess.ts	The three worker steps (S3, SharePoint, Teams) with retries and the postprocess.json status.
postprocessQueue.ts	Hand-off from the test: writes job files, starts the worker, copies its log into the test output every 3 s, waits for it at the end.
postprocessWorker.ts	The worker process: takes jobs one at a time, prefixes each log line with the branch name.
postprocessCli.ts	npm run post: run the post-processing of one finished branch by hand.
utils/ - delivery	
sharepointSession.ts	SharePoint upload with your own sign-in (REST). Sign-in itself: spLogin.ts.
sharepointGraph.ts	SharePoint upload through Microsoft Graph (needs an app registration); also holds the URL parser and the day-folder naming.
sharepoint.ts	Old OneDrive-shortcut copy (SHAREPOINT_DIR).

File	What it does
spLogin.ts	npm run sp:login.
teams.ts	Builds and posts the Teams cards (with retries). teamsWorker.js sends a failure card with a shrunk screenshot in a separate process so it can never hold up the run.
awsConsole.ts	Old AWS-portal login helper. Not used by the current flow.
queue-server/ (optional)	
server.js	The branch queue and its web page, fair rounds, stuck-run watchdog with auto-recovery.
mirror.js, freezeTracker.js, keepAwake.js	Copies log lines to /output, records why a run froze (PC sleep, killed process), keeps Windows awake.
public/index.html, public/output.html	The queue page and the live-output page.
qa-eks-mentor/ (the AI model project)	
analyze.py	Pulls the QA EKS logs of one run and tool (kubectl), finds why the tool never triggered with rules, lets the model write the summary, and writes Analysis_Report_<tool>-<run>.pdf and .md into reports/.
Modelfile, MENTOR_PROMPT.md	Ollama definition of the model (Qwen2.5 3B, temperature 0.1) and its rules: cite a log line for every claim, otherwise say "Insufficient evidence".
training/ (prepare_dataset.py, train_colab.py, the Colab notebook, dataset.jsonl)	How the model was fine-tuned (QLoRA on a free Colab GPU) and the training examples (2 so far).
examples/, requirements.txt	Sample facts files and a sample report; the Python packages.
README.md, QA_EKS_MENTOR_DOCUMENTATION.md, MODEL_TRAINING_GUIDE.md	Its own documentation.
Created while you work (not in git)	
.auth/	Saved sessions: Testable (user.json, user-2.json), SharePoint (sharepoint-session.json, sharepoint-token.json).
results/	Everything a run produces (section 10).
test-results/, playwright-report/ (and ..-acct2)	Playwright's own output and HTML report.
queue-server/data/	Queue state, logs, heartbeat and freeze-tracker files.
core/, __pycache__/	Compiled Python cache files left from another branch. Not used; safe to delete.

9 .env reference

every setting the scripts read

Create the file **.env** in the project folder (copy .env.example). One setting per line, **NAME=value**, no spaces around the equals sign. **The values in .env always win**: a variable typed in the terminal is ignored if the same name is in .env. (BRANCHES, REPO, ACCOUNT, LANES and PARALLEL_* are set in the terminal, not in .env.)

Must have

Variable	What to put	Default / notes
TEST_ENV	**qa** or **dev**: which Testable site and S3 bucket to use.	qa
TESTABLE_EMAIL TESTABLE_PASSWORD	The Testable account for QA (account 1).	required for qa
TESTABLE_EMAIL_DEV TESTABLE_PASSWORD_DEV	The Testable account for DEV.	required when TEST_ENV=dev
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN	Temporary keys of the QA-S3-Readonly role (AWS access portal, Access keys). Needed for the S3 check.	expire after hours to about a day; without them the S3 step is skipped

Second account (two-account mode)

Variable	What to put	Default / notes
TESTABLE_EMAIL_2 TESTABLE_PASSWORD_2	The second Testable account (DEV: TESTABLE_EMAIL_DEV_2 / TESTABLE_PASSWORD_DEV_2). More accounts: _3, _4 ...	needed for npm run parallel with two lanes
ACCOUNT (terminal)	2 = run this terminal with the second account (its own session and results folder).	empty = account 1
LANES (terminal)	Number of lanes for npm run parallel, e.g. 1.	all accounts found
PARALLEL_STAGGER_SEC (terminal)	Seconds between lane starts.	20

Delivery (optional but recommended)

Variable	What to put	Default / notes
TEAMS_WEBHOOK_URL	The Workflows webhook of the Teams channel ("Send webhook alerts to a channel"). Keep it secret.	empty = no Teams messages
SP_URL	Address-bar URL of another SharePoint parent folder.	Automation Evidences Dump (built in)
SP_TENANT_ID SP_CLIENT_ID	From the Entra app registration. Only for the Graph method; the sign-in session method needs neither.	empty = session method
SHAREPOINT_DIR SHAREPOINT_URL	Old OneDrive-shortcut method: the local folder, and the folder's address-bar URL for the Teams link. Change them every day.	not needed with sp:login
GITHUB_TOKEN	Lets bug reports read the branch's trigger file from private repos.	optional

Run behaviour (optional)

Variable	What it does	Default
POSTPROCESS	**background**: worker beside the next branch. **inline**: everything in the test, one branch after the other.	background
GATE_SHOTS	**on** takes a screenshot of every gate. Off still reads all gate values.	off
DUMMY_PR	**yes**: push a README comment to every branch first so a new commit is analysed. Needs git signed in with write access.	no
RUN_TIMEOUT_MIN	How long to wait for one analysis to finish.	35
REPO_SYNC_CAP_MIN	How long the repository sync may take before the branch is marked as an environment problem.	14

Variable	What it does	Default
LANGUAGE	Force the language(s) for the S3 check, e.g. javascript,typescript. Normally detected.	detected
LOGIN_MODE	**normal** (email + password) or **okta** (Sign in with SSO). OKTA_EMAIL / OKTA_PASSWORD, if set, are used for the Okta route.	normal
HEADLESS, PAGE_SCALE	HEADLESS=1 hides the browser. PAGE_SCALE (e.g. 0.9) shows more of the page in screenshots.	visible, 1
BUG_REPORTS	**off** turns bug drafts off.	on
KEEP_AWAKE	**off** stops the script from keeping Windows awake.	on
PY_IGNORE_TOOLS, JS_..., TS_..., CS_..., JAVA_IGNORE_TOOLS	Comma-separated tools the validator should not expect.	none
S3_BUCKET, AWS_REGION, BASE_URL	Override the bucket (QA: testable-qa-artifacts-759498976252, DEV: dev-bucket-us2), region (us-east-2) or the site address (default https://<env>-app.testable.cloud).	follow TEST_ENV
AWS_PROFILE	Use an AWS SSO profile instead of the three keys.	none
OLLAMA_HOST, OLLAMA_QA_MODEL	Only for the optional QA EKS Mentor: address of the local Ollama and the model name.	http://localhost:11434, qa-eks-mentor-trained:latest
EKS_MENTOR_DIR	Folder whose reports\ holds the ready analyze.py reports that bug drafts print.	also qa-eks-mentor\ and F:\Ollama\mentor
EKS_AWS_ACCESS_KEY_ID EKS_AWS_SECRET_ACCESS_KEY (EKS_AWS_SESSION_TOKEN)	The team's read-only QA EKS access key (an IAM key, not the AWS portal user name and password). With both set, the QA EKS analysis runs by itself. Keep it in .env only.	empty = off
EKS_AUTO, EKS_AWS_PROFILE, EKS_CLUSTER_NAME, EKS_REGION, EKS_PYTHON, EKS_ANALYZE_MIN, EKS_KUBECTL, EKS_AWS_CLI	on or off; AWS profile for your own SSO session; cluster name; region; the python command; minutes at most; full paths of kubectl and aws when they are not on the PATH.	on/off, qa-eks-ro, qa-eks, us-east-2, python, 8

Branch input and advanced

Variable	What it does
BRANCHES, REPO, BUILDS_FILE	The branch list: BRANCHES="link1,link2" (full GitHub /tree/ links), or REPO plus branch names, or another JSON file. Set them in the terminal, not in .env.
S3_RESULTS_DIR, S3_FORCE	For npm run s3: which results folder to re-check, and 1 to redo a branch that was already checked.
RUN_DIR	The output folder of the run. Set by the config; the queue server and the two-account launcher set it for their runs.
POSTPROCESS_WAIT, POSTPROCESS_ECHO, QUEUE_NO_MIRROR	Internal switches set by the two-account launcher for each lane. Do not set them yourself.
SP_DEBUG=1	Prints every SharePoint call (for finding problems).
QUEUE_PORT, QUEUE_CODE, QUEUE_ROUND_MAX, QUEUE_CMD, QUEUE_RECOVER_START_MIN, QUEUE_RECOVER_MAX_MIN, QUEUE_STUCK_MIN	Queue server: port (3000), access code for the page, most people per round (10), the command a round runs (npm test), and the watchdog times (6, 50 and 8 minutes).

A safe starting .env (replace the placeholders, never paste real secrets into a document)

```
TEST_ENV=qa
TESTABLE_EMAIL=your.name@company.com
TESTABLE_PASSWORD=your-password
# second account, only for npm run parallel
TESTABLE_EMAIL_2=second.account@company.com
TESTABLE_PASSWORD_2=second-password

AWS_ACCESS_KEY_ID=ASIA...
AWS_SECRET_ACCESS_KEY=...
AWS_SESSION_TOKEN=...
AWS_REGION=us-east-2

TEAMS_WEBHOOK_URL=https://...
POSTPROCESS=background
GATE_SHOTS=off
DUMMY_PR=no
```

10 Result folders

what every output file is

```
results/merge-flow_2026-10-29_1127/          one folder per run (two accounts: ..._acct1, ..._acct2)
  summary.json                               status and minutes of every branch
  postprocess.log                            the background worker's full log
  evidence/, evidence.jsonl                  step screenshots and the pass/fail record
  .postprocess/                              job files of the worker (section 5.1)
  <branch name>/                             one folder per branch
    01_repo_structure.png                    coverage preview screenshots
    02_what_we_will_analyze.png
    03_uncovered_areas.png
    04_pr_confidence_score.png               the PR score
    05_execution_status.png                  the execution status tick
    gates/                                   gate screenshots (only with GATE_SHOTS=on)
    ui-values.json                           every gate value read from the page
    repo-details.json                        build type, language, language version
    run-details.json                         run id, branch id, commit (the S3 path)
    taxonomy-gate-<run>.html                 the report as downloaded
    taxonomy-gate-...-with-ui-values.html    the finished report (UI + S3 columns)
    s3-folders.json, s3-files.json           what the run wrote to S3, file excerpts
    s3-evidence/                             a copy of every file of every tool
    s3-check.json                            every verdict, validator summaries, defects
    bugs/                                    Defect_Report_*.pdf, JIRA_*.jira.md, bugs.json
    qa-eks-analysis/                         QA EKS analysis reports (PDF + .md) of the defective tools
    postprocess.json                         status of S3, SharePoint and Teams
    errors/                                  failed branch only: failure.png, network.json,
                                               site-error.txt, teams-notify.log
    RUN-MISMATCH.txt                         only if the run on the site was another branch
    sharepoint-copy-errors.txt               only if some files could not be uploaded
results/parallel_<date>_<time>_summary.json  two-account mode: every branch, its account and status
```

The same branch in SharePoint

General / QA-Testable / Testable Whitebox Tool Execution / Automation Evidences Dump / 29-10-2026 / <branch name> / followed by the whole branch folder above, including s3-evidence and bugs.

How to read the finished report

Part of the report	Meaning
Box at the top	The S3 path of the run, a link to the AWS console, and one line per language: tools expected, how many OK, clean, empty, false-empty, failed, missing (Planning Defect), tools that ran when they should not have, and foreign-language tools.

Part of the report	Meaning
UI value column	The value the Confidence Engine page shows. Same as the report: plain. Different: red, with the report's value in the tooltip. Not found on the page: a grey dash.
Tool / JSON and S3 comment columns	The tool's folder and files in S3 (a saved copy and a link to the AWS console), the PASS / EMPTY / FAIL tag, and the note. A "secondary" tag means the primary tool's folder was missing and the secondary tool was used.

11 Troubleshooting

symptom, likely cause, what to do

What you see	Likely cause	What to do
All tests skipped, "No branches configured"	builds.json is empty and BRANCHES is not set	Add the branches (section 7.6) and refresh the test list
Login fails in auth.setup.ts	Wrong TESTABLE_EMAIL / PASSWORD, or the site expects Okta	Run npm run setup; check .env; try LOGIN_MODE=okta
Branch BLOCKED: "Branch unavailable"	The site's repository list is out of date (the branch exists on GitHub)	Run that branch again later; check the name on GitHub
Branch BLOCKED: "Analysis is blocked on ..."	The site refuses the analysis (for example a missing language version)	The branch stops by itself after about 10 s; read errors\site-error.txt and fix the repository
ENVIRONMENT: repo sync did not finish	The Testable QA backend is slow or down	Try again later; raise REPO_SYNC_CAP_MIN if needed
S3 step: ExpiredToken or "AWS keys expired"	The temporary AWS keys ran out	Paste fresh keys in .env, then npm run post -- "<branch folder>"
S3 or SharePoint or Teams: ENOTFOUND, fetch failed	Short network or DNS outage	Automatic retries after 30 s, 90 s, 3 min; if it still failed use npm run post
SharePoint: "sign-in expired" or "not signed in"	The saved session ended or was never made	Run npm run sp:login
SharePoint: some files failed	File over 240 MB, or a very long path	See sharepoint-copy-errors.txt in the branch folder
No Teams message for a passed branch	Webhook empty or wrong, or the network was down	Check postprocess.json (teams: failed?), fix .env, npm run post
No [post] lines after a branch passed	POSTPROCESS=inline in .env, or npm install was not run (tsx missing)	Set POSTPROCESS=background; run npm install
npm run parallel says "1 account lane(s)"	TESTABLE_EMAIL_2 or TESTABLE_PASSWORD_2 is missing or misspelled in .env	Add both lines exactly as written and save the file
npm run parallel: "Do not set ACCOUNT"	ACCOUNT is set in your terminal	Remove it: Remove-Item Env:ACCOUNT
Lane 2 "did not start" twice, then stops	The second login is wrong, or the site does not allow a second run	Test it: $env:ACCOUNT="2" then npm run setup; check the second account by hand
The run stopped by itself while the PC was in use	Editing .env or playwright.config.ts restarts the VS Code Playwright server (Test Explorer runs); or the PC slept	Do not touch those files during a run; keep the PC awake; use a terminal for long runs
Browser is not visible in the Test Explorer	"Show browser" is off	Turn it on in the Playwright section
Bug drafts have no AI diagnosis block	There is no ready report -.md for that tool and run in the mentor folder (direct model answers are not printed)	Run analyze.py for that run and tool, then npm run post -- "<branch folder>" (section 12)
AI says "Insufficient evidence"	No log lines reached the model (it only gets run facts when asked directly), or the EKS logs are too old	Run analyze.py right after the run so a ready report exists (section 12)
RUN-MISMATCH.txt in a branch folder	The run opened on the site belongs to another branch	Do not trust that folder; run the branch again
A terminal value is ignored	.env wins over the terminal	Change the value in .env

First place to look
For any problem after a branch finished: open **<branch folder>/postprocess.json**. It tells you which of S3, SharePoint and Teams worked and shows the error text of the one that did not. The full story is in **postprocess.log** in the run folder.

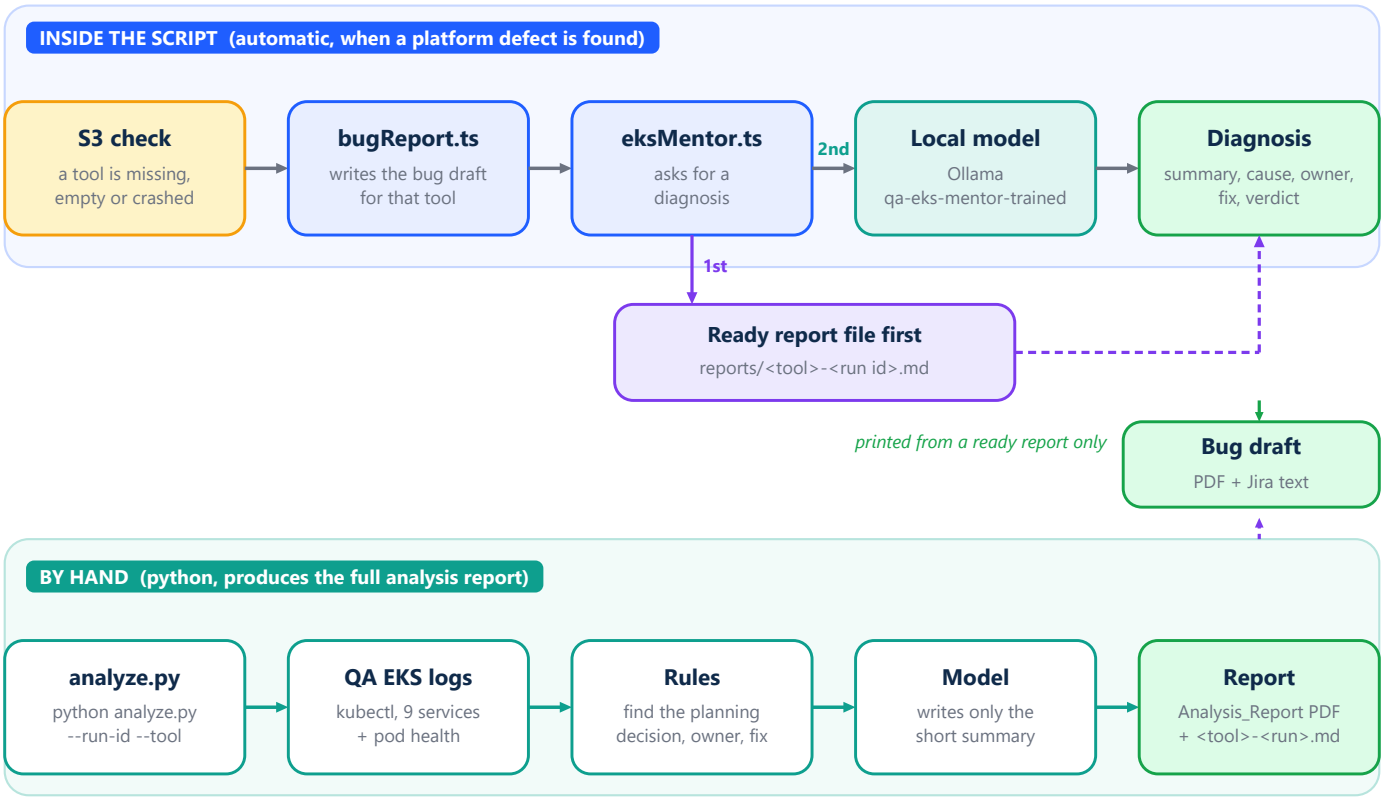
12 QA EKS Mentor (AI log analysis)

a local model that explains why a tool did not trigger

When the S3 check finds a tool that never ran, the bug draft used to say only "not triggered". Finding the real cause meant searching the logs of nine services on the QA EKS cluster by hand, about 30 to 45 minutes per tool. The **QA EKS Mentor** does that search and writes the explanation. It is a small AI model that runs **on your own PC** through Ollama: no cloud service, no cost per use, and no company data leaves the machine.

Fact	Detail
The model	**qa-eks-mentor-trained:latest** in Ollama (about 1.9 GB). Base model Qwen2.5 3B Instruct, fine-tuned with QLoRA on a free Google Colab GPU, packed as a 4-bit GGUF file (Q4_K_M). Temperature 0.1.
Checked on this PC	Ollama is running here and lists qa-eks-mentor-trained (created 29 Oct, 13:30) next to an older qa-eks-mentor. The Ollama address is http://localhost:11434.
Its rules (Modelfile and MENTOR_PROMPT.md)	Every claim in the root cause must quote a log line (time, logger, message). If there is no log line for the run it must answer **Verdict: Insufficient evidence** and say which checks to run next. Evidence from other runs may only appear as an unverified hypothesis.
What it writes	A report with eight headings: Summary, Run Metadata, Evidence from QA EKS, Root Cause, Hypotheses, Recommended Fix (owner and action), QA Acceptance Criteria, Gaps. Verdicts: Planning, Execution, Environment, Repo issue or Insufficient evidence.

How it plugs in



Inside the script: for every platform defect **bugReport.ts** calls **eksMentor.ts**. It first looks for a ready report file named *<tool>-<first 8 characters of the run id>.md* in the mentor folder (**F:\Ollama\mentor\reports**). If there is none, it asks the local model directly (12 seconds at most per tool) with the run facts only. **The bug draft prints the diagnosis only from a ready report**, because a report is built from real log lines; a direct answer carries no log lines and is left out. Anything offline returns nothing and the bug draft is written exactly as before. **By hand: analyze.py** reads the cluster logs and writes the full report; the model only writes its short summary, the rules decide the cause.

How the diagnosis is printed, and what is still open
1. Printed from ready reports only. When the folder **<mentor folder>\reports** holds *<tool>-<first 8 characters of the run id>.md* written by analyze.py, the defect PDF (page 2) and Jira section 4 get a block *AI root-cause diagnosis* with verdict, summary, root cause, the QA EKS log line, owner and fix. Without such a report the bug draft is byte for byte the same as before. **2. Direct answers are not printed.** In a test the model, asked without log lines, invented a log line dated 2024. A diagnosis without evidence must not reach a defect report. **3. The folder** defaults to F:\Ollama\mentor; set **EKS_MENTOR_DIR** for another PC. analyze.py writes into the reports/ folder next to itself, so run it from that folder. The older .md files in F:\Ollama\mentor\reports are raw model dumps and show nothing. **4. The model learned from 2 examples.** Treat it as a summariser until it has 50 to 100 real examples.

Automatic analysis (no typing)

With the team's read-only key in **.env** (EKS_AWS_ACCESS_KEY_ID and EKS_AWS_SECRET_ACCESS_KEY), or **EKS_AUTO=on** with your own aws sso session, the worker runs **analyze.py** by itself just before the bug drafts are written, for the defective tools that have no ready report. Steps: use the cluster key (the S3 keys in .env are never used for the cluster), check the kubectl context *qa-eks-ro* (created with aws eks update-kubeconfig when missing), check that the cluster answers, run analyze.py (8 minutes at most), then print the ready reports in the bug drafts. Any problem prints one line and the run carries on. It runs in the background worker, so the next branch is not delayed. The analysis reports it writes (*Analysis_Report_<tool>-<run>.pdf* and the .md) are **copied into the branch folder** (**qa-eks-analysis/**), so the SharePoint upload takes them along. The same happens for reports you made by hand, as long as they are in the mentor's reports folder when the bug drafts are written (**npm run post** redoes it).

Use an access key, and check the network
The cluster key must be an **IAM access key** (id and secret). A user name and password, the sign-in of the AWS access portal, cannot be used by kubectl and the scripts do not type it for you. Share the key only through a private channel, never in the repository, a document or a chat. The **cluster must also be reachable from your PC**: on the PC used for this guide the AWS login works but the cluster endpoint timed out, so the analysis would be skipped. That points to a network or VPN allow-list, not to credentials.

Set it up on another PC

#	Do this	Command or note
1	Install Ollama from ollama.com and Python 3.10 or newer.	
2	Get the model file (about 1.9 GB, not in git) from its owner, or build it with MODEL_TRAINING_GUIDE.md, and put it next to the Modelfile.	Qwen2.5-3B-Instruct.Q4_K_M.gguf
3	Create the model in Ollama.	`ollama create qa-eks-mentor-trained -f Modelfile`
4	Check that it is there.	`ollama list`

5	For analyze.py: install the packages and the cluster tools, then sign in (private window).	`pip install -r requirements.txt`
		AWS CLI v2 + kubectl, profile and context **qa-eks-ro**
		`aws sso login --profile qa-eks-ro --no-browser`

6	Run the analysis right after the Playwright run (the cluster keeps only the current and previous container logs).	`python analyze.py --run-id <run-id> --tool crosshair,pymcdc` add **--no-llm** for a rules-only report
7	So that bug drafts find the report, run analyze.py from the mentor folder, or point the script at it. Then redo the bug drafts of the branch.	EKS_MENTOR_DIR (folder that contains reports\), OLLAMA_HOST, OLLAMA_QA_MODEL (section 9) `npm run post -- "<branch folder>"`

The mentor's own documents: **qa-eks-mentor/README.md** (usage), **QA_EKS_MENTOR_DOCUMENTATION.md** (architecture and method), **MODEL_TRAINING_GUIDE.md** (training), and the PDF **QA_EKS_Mentor_Architecture_and_Methodology.pdf**. It is uncommitted work, and **bugReport.ts and eksMentor.ts must be committed together**, because bugReport.ts imports eksMentor.ts.

13 Cheat sheet

commands, words and the optional queue

Command	What it does
`npm test`	The full single-account run: login, every branch, logout, wait for the worker.
`npm run parallel`	Two-account run (working copy): all branches on all accounts in .env.
`npm run setup`	Only the Testable login (saves .auth/user.json). Useful to test your credentials. With $env:ACCOUNT="2" it tests the second account.
`npm run sp:login`	One-time SharePoint sign-in for you.
`npm run post -- "<branch folder>" --repo owner/name`	Repeats S3 check, bug drafts, SharePoint and Teams for one finished branch.
`npm run s3`	Old way: re-checks S3 for a results folder (S3_RESULTS_DIR, S3_FORCE=1). Normally not needed.
`npm run queue`	Starts the optional branch queue server.
`python analyze.py --run-id <id> --tool <tool>`	In qa-eks-mentor/: the AI log analysis of one tool of one run (needs kubectl access and Ollama).

Words used in this guide

Word	Meaning
Branch folder	results/<run>/<branch name>/ : everything for one branch.
Main flow / browser lane	The 15 steps done with Playwright on the Testable site.
Worker / background lane	The separate process that does S3, SharePoint and Teams without a browser.
Lane	In two-account mode: one Testable account with its own browser, results folder and worker.
Hand-off	The moment a passed branch is given to the worker (a job file is written).
Gate / technique / classification	The rows of the taxonomy report: a technique (for example Cyclomatic Complexity) has classifications, each with a value.
Primary / secondary tool	The tool expected for a technique, and its back-up when the primary tool's folder is missing.
QA EKS Mentor	The local AI model (qa-eks-mentor-trained in Ollama) and its scripts that explain why a tool did not trigger, using the QA EKS cluster logs.
Platform defect	A problem in the Testable platform (tool not planned, empty report, crash, wrong language) as opposed to a problem in the customer's repo.
Blocked / environment / failed	The three ways a branch can end without passing: the site refused, the site or network was unavailable, or a step failed.

One-minute routine

Before	During	After
1. Fresh AWS keys in .env 2. Branches in builds.json or BRANCHES 3. npm run sp:login (once)	4. npm test or npm run parallel 5. Do not touch .env or the config 6. Watch [post] [branch] lines	7. Teams card + SharePoint folder 8. postprocess.json if something is missing 9. git checkout builds.json

The optional branch queue

When many people add branches, run **npm run queue** and open **http://localhost:3000** (the team address is printed). Everybody adds their GitHub branch links with their name. The owner types **start** in the terminal. A round takes one branch from each waiting person, the person served longest ago first, and runs it with the normal flow. Set **QUEUE_CODE** in .env before sharing the page outside the office network. A run that freezes at the start of a branch is stopped and restarted by itself. This queue serves one account; for two accounts use npm run parallel.



The owner types start | pause | stop | clear | list | remove <id> | status in the terminal that runs the server.

About this document
It describes the dev branch at a78fbb1 + working copy. If you change a file in the file map, update the matching section here. The script prints what it does; when in doubt, trust the log lines and postprocess.json over this guide.